

MANUALE DELLO SVILUPPATORE — SKILLSHARE

CORSO DI INGEGNERIA DEL SOFTWARE

INFORMATICA PER IL MANAGEMENT, A.A. 2025/2026

REPOSITORY: [PROGETTO_SKILLSHARE](#)

COMPOSIZIONE DEL GRUPPO: [TOMMASO PRANDINI](#), [FEDERICO MALSERVIGI](#), [ALESSIA SIRIANNI](#), [GABRIELE BUFALO](#).

SkillShare è una piattaforma di baratto di competenze: gli utenti pubblicano annunci in cui offrono una competenza e indicano cosa desiderano in cambio, si scambiano richieste, dialogano in chat e si recensiscono a scambio concluso.

Questo manuale è rivolto a uno sviluppatore che riceve il repository per la prima volta e non conosce il progetto.

1. PREREQUISITI

1.1 SOFTWARE RICHIESTO

Strumento	Versione	Quando serve
Java JDK	17 o superiore	Sviluppo locale e compilazione
Apache Maven	3.8 o superiore	Build e test
Docker + Docker Compose	recenti	Avvio con container
Git	qualsiasi	Clone del repository
Browser	recente	Uso dell'applicazione

- Per il solo avvio con Docker bastano Git e Docker: JDK e Maven sono già presenti nell'immagine.
- Per lavorare sul codice servono JDK e Maven installati sulla macchina.

1.2 INSTALLAZIONE

- Le procedure di installazione di JDK e Maven per Ubuntu/Debian e per Windows sono riportate nel README.md alla radice del repository, comprese la configurazione di JAVA_HOME e l'aggiunta di Maven al Path.
- Su Windows conviene usare i comandi winget indicati nel README: evitano la configurazione manuale delle variabili d'ambiente.

1.3 VERIFICA DELL'AMBIENTE

```
java -version    # deve riportare la versione 17 o superiore
mvn -version     # deve riportare la 3.8 o superiore e la JDK 17
docker --version
docker compose version
```

- Se mvn -version riporta una JDK diversa dalla 17, JAVA_HOME punta a un'altra installazione: correggerla prima di procedere.
- La porta **8080** deve essere libera: è quella su cui l'applicazione risponde sia con Docker sia in modalità sviluppo.

2. BUILD E AVVIO

2.1 PRIMA COMPILAZIONE

Dalla radice del repository:

```
mvn clean install
```

- Il primo lancio è lento: Maven scarica le dipendenze e GWT compila l'intero frontend in JavaScript. I lanci successivi sono molto più rapidi.
- La compilazione GWT è la fase più lunga: se il log sembra fermo su di essa, sta lavorando.
- **Prima di eseguire il codice lanciare una volta il seguente comando per popolare il progetto con i dati demo**

```
mvn exec:java -pl skillshare-server
```

2.2 AVVIO CON DOCKER

È il percorso consigliato per vedere l'applicazione funzionante senza configurare nulla.

```
docker compose up -d --build # costruisce l'immagine e avvia il container  
docker compose down        # arresta e rimuove il container
```

- Applicazione raggiungibile su <http://localhost:8080/>.
- I dati (database e foto profilo) sono resi persistenti su disco tramite un volume, quindi sopravvivono al riavvio del container.
- Per azzerare i dati: arrestare i container e cancellare la cartella dei dati persistenti indicata nel README.md. **L'operazione è irreversibile.**

2.3 AVVIO IN MODALITÀ SVILUPPO

Servono **due terminali distinti**, aperti sulla radice del repository.

Terminale 1 — GWT Code Server, che ricompila il frontend a ogni modifica:

```
mvn gwt:codeserver -pl *-client -am
```

Terminale 2 — server Jetty, che espone il backend:

```
mvn jetty:run -pl *-server -am -Denv=dev
```

- Applicazione raggiungibile su <http://127.0.0.1:8080/> quando entrambi i processi sono attivi.

- Ciclo di lavoro: modificando una classe di interfaccia basta ricaricare la pagina nel browser, perché il Code Server ricompila da solo; modificando una classe lato server Jetty rileva il cambiamento e ridistribuisce l'applicazione.
- L'ordine di avvio conta: se Jetty parte prima che il Code Server abbia terminato la prima compilazione, la pagina resta bianca finché non si ricarica.

2.4 VERIFICA CHE TUTTO FUNZIONI

Eseguire nell'ordine:

1. Aprire l'URL: compare la schermata di benvenuto con il marchio SkillShare e i pulsanti **Registrati** e **Login**.
2. Accedere con l'utente demo (§6) oppure registrare un nuovo account.
3. Dopo l'accesso compare la barra di navigazione con le voci **Profilo**, **I miei annunci**, **Marketplace**, **Richieste**.
4. Aprire **Marketplace**: la lista degli annunci si carica senza messaggi di errore.
5. Aprire **Profilo**: vengono mostrati nome, biografia e, se presenti, valutazione media e recensioni ricevute.

Se uno di questi passaggi fallisce, vedere il §7.

3. ESECUZIONE DEI TEST

Il progetto usa **JUnit 5**. Tutte le classi di test vivono nel modulo server.

3.1 COMANDI

```
mvn test                # tutti i test del progetto
mvn test -pl *-server   # solo il modulo server
mvn test -pl *-server -Dtest=NomeDellaClasseTest # una singola classe
mvn clean test          # ricompilando tutto da zero
```

3.2 LE CLASSI DI TEST

Classe di test	Verifica
UtenteDatabaseTest	Registrazione, formato di email e password, duplicati, verifica delle credenziali
ProfileServiceImplTest	Lettura e aggiornamento del profilo
ProfiloVisibilitaTest	Quando un profilo va mostrato in sola lettura
ValidatoreFotoProfiloTest	Regole di accettazione delle immagini caricate
ArchivioFotoProfiloTest	Nomi dei file, salvataggio e recupero delle immagini
AnnuncioDatabaseTest	Pubblicazione, campi obbligatori, elenco per utente

Classe di test	Verifica
AnnuncioDatabaseModificaRimuoviTest	Modifica e rimozione, controllo del proprietario
AnnuncioServiceImplTest	Inoltro delle operazioni sugli annunci
MarketplaceServiceImplTest	Filtri, ordinamenti, arricchimento con autore e valutazione
MarketplaceRicercaAvanzataTest	Ricerca testuale sui campi selezionabili
RichiestaScambioDatabaseTest	Salvataggio, accettazione, rifiuto, controllo di proprietà
RichiestaScambioServiceImplTest	Regole di invio, tra cui il divieto di scambio con sé stessi
RichiestaScambioCompletabileTest	Condizioni che rendono una richiesta completabile
MessaggioDatabaseTest	Invio dei messaggi e accesso ai soli partecipanti
ChatServiceImplTest	Inoltro delle operazioni di chat
RecensioneDatabaseTest	Salvataggio delle recensioni e calcolo della media
RecensioneServiceImplTest	Regole di scrittura e lettura delle recensioni
ReputazioneServiceImplTest	Valutazione media e recensioni ricevute
StelleTest	Conversione dei voti nella rappresentazione a stelle

3.3 LETTURA DEI RISULTATI

- A fine esecuzione Maven stampa il riepilogo Surefire nella forma Tests run: N, Failures: N, Errors: N, Skipped: N.
- **Failure:** un'asserzione non è stata soddisfatta, cioè il codice ha fatto qualcosa di diverso da quanto atteso.
- **Error:** è stata sollevata un'eccezione non prevista dal test, spesso segno di un problema di ambiente e non di logica.
- I dettagli completi, stack trace inclusi, si trovano nella cartella target/surefire-reports/ del modulo server: un file di testo e uno XML per ogni classe di test.
- La build si ferma al primo modulo con test falliti: Tests run conta solo quelli eseguiti fino a quel punto.

4. FUNZIONALITÀ REALIZZATE

Ogni funzionalità segue la stessa catena: una schermata *Gui lato client chiama un servizio remoto, il servizio (*ServiceImpl) coordina l'operazione e delega a una classe di accesso ai dati (*Database), che applica le regole e persiste su MapDB. Gli oggetti scambiati tra le due parti sono i DTO (UtenteDTO, AnnuncioDTO, RichiestaScambioDTO, MessaggioDTO, RecensioneDTO).

4.1 REGISTRAZIONE E ACCESSO

- **Registrazione:** RegistrazioneGui → RegistrazioneServiceImpl.registraUtente() → UtenteDatabase.registra().
 - Controlli applicati: formato dell'email, password di almeno 8 caratteri con una maiuscola, una minuscola, una cifra e un simbolo tra @\$%^&+=!, unicità dell'email.
 - La password non viene mai salvata in chiaro: UtenteDatabase.hashPassword() ne registra l'impronta SHA-256.
- **Accesso:** LoginGui → AccessoServiceImpl.login() → UtenteDatabase.verificaCredenziali(), che confronta l'impronta della password inserita con quella salvata.
 - Con credenziali errate il servizio non solleva un'eccezione ma restituisce un UtenteDTO che porta il messaggio d'errore: evita al client una risposta di errore HTTP e permette di mostrare il messaggio nel form.
- Dopo l'accesso, NavBar è la barra condivisa da tutte le schermate e porta alle quattro sezioni principali.

4.2 PROFILO E FOTO PROFILO

- **Visualizzazione:** ProfiloGui → ProfileServiceImpl.getProfilo() → UtenteDatabase.getProfilo(). La stessa schermata mostra sia il proprio profilo sia quello altrui: ProfiloVisibilita.soloLettura() decide se nascondere le azioni personali come "Modifica Profilo". In caso di identificativo mancante resta in sola lettura, perché mostrare per errore il pulsante sul profilo di un altro sarebbe più grave che nascondere sul proprio.
- **Modifica:** ModificaProfiloGui → ProfileServiceImpl.updateProfile() → UtenteDatabase.aggiornaProfilo(), che aggiorna solo biografia, foto e competenze, preservando credenziali e dati anagrafici.
- **Foto profilo:** il caricamento non passa dal meccanismo RPC (Chiamata di Procedura Remota), che serializza solo oggetti Java, ma da un form multipart.
 - FotoProfiloUploadServlet riceve il file, ValidatoreFotoProfilo ne verifica nome, tipo e dimensione (JPG o PNG, massimo 2 MB), ArchivioFotoProfilo lo salva sostituendo l'eventuale immagine precedente dello stesso utente.
 - Il percorso pubblico dell'immagine viene scritto nel campo photoUrl del profilo; FotoProfiloServlet serve poi il file al browser.
 - Nomi dei campi, prefissi di risposta (OK|, ERRORE|) e limite di dimensione sono definiti una volta sola in ProtocolloUploadFoto, condiviso tra client e server (§5.6).

4.3 GESTIONE DEGLI ANNUNCI

- **Pubblicazione:** NuovoAnnuncioGui → AnnuncioServiceImpl.pubblica() → AnnuncioDatabase.pubblica(), che verifica i campi obbligatori (titolo, competenza offerta, disponibilità, controprestazione), assegna un identificativo univoco e registra la data di creazione.
- **Gestione:** MieiAnnunciGui → AnnuncioServiceImpl.modifica(), rimuovi(), cambiaDisponibilita().

- Ogni operazione riceve l'identificativo di chi la richiede e viene rifiutata se non coincide con il proprietario dell'annuncio.
- Un annuncio con uno scambio in corso non è modificabile: la condizione viene verificata in `AnnuncioDatabase`, che consulta le richieste di scambio.
- **Sospeso e rimosso** sono cose diverse: un annuncio sospeso resta visibile al proprietario in "I miei annunci" ma sparisce dal marketplace, mentre uno rimosso è cancellato.

4.4 MARKETPLACE, RICERCA E ORDINAMENTO

- `MarketplaceGui` → `MarketplaceServiceImpl.listaAnnunci()` per l'elenco, `cercaAnnunci()` per la ricerca.
- La ricerca testuale accetta un insieme di campi su cui cercare, rappresentati dall'enum `CampoRicerca` (titolo, competenza, controprestazione, autore): un annuncio è incluso se corrisponde in **almeno uno** dei campi selezionati.
- Gli annunci sospesi vengono scartati prima di ogni filtro.
- Prima di essere restituito al client ogni annuncio viene arricchito con il nome completo dell'autore e con la sua valutazione media, che non sono salvati nell'annuncio ma ricavati da `UtenteDatabase` e `RecensioneDatabase`. Se l'autore non è più registrato resta visibile la sua email.
- Ordinamenti disponibili: alfabetico per titolo (`ordinaPerTitolo`) e per valutazione dell'autore decrescente (`ordinaPerRatingAutoreDesc`); in assenza di ordinamento esplicito gli annunci sono in ordine cronologico decrescente.

4.5 RICHIESTE DI SCAMBIO E CHAT

- `RichiesteGui` → `RichiestaScambioServiceImpl`, che espone l'invio della richiesta, gli elenchi di richieste ricevute e inviate, e le azioni `accetta()`, `rifiuta()`, `completa()`.
- Il ciclo di vita di una richiesta è descritto dall'enum `StatoRichiesta`: `PENDING` alla creazione, poi `ACCEPTED` o `REJECTED` per decisione del proprietario dell'annuncio, infine `COMPLETED` quando lo scambio è concluso.
- Due regole vivono nel servizio e non nel client: il proprietario dell'annuncio viene ricavato dall'annuncio stesso lato server, e non accettato come parametro; una richiesta sul proprio annuncio viene rifiutata.
- **Chat**: `ChatGui` → `ChatServiceImpl` → `MessaggioDatabase`. I messaggi sono legati a una richiesta di scambio e la lettura è consentita ai soli due partecipanti, verifica svolta lato server.

4.6 RECENSIONI E REPUTAZIONE

- **Scrittura**: `CompletamentoRecensioneGui` → `RecensioneServiceImpl.lascia()` → `RecensioneDatabase`, al termine di uno scambio.
- **Recensioni di un annuncio**: `RecensioniAnnuncioGui` → `RecensioneServiceImpl.recensioniPerAnnuncio()`, che arricchisce ogni recensione con il nome di chi l'ha scritta.
- **Reputazione sul profilo**: `ReputazioneServiceImpl.ratingMedio()` e `recensioniRicevute()` alimentano la sezione reputazione di `ProfiloGui`.
- La resa grafica dei voti è centralizzata in `Stelle`, che produce una striscia sempre lunga cinque simboli e formatta la media con una cifra decimale. La media viene calcolata con aritmetica intera perché in GWT la conversione di un `double` passa da JavaScript e trasformerebbe 4.0 in "4".

- Il riquadro della singola recensione è `RecensioneItem`, condiviso tra le recensioni di un annuncio e il profilo pubblico, così le due schermate restano identiche senza duplicare il codice di disegno.

5. DESIGN PATTERN ADOTTATI

I pattern elencati sono solo quelli riscontrabili nel codice scritto dal gruppo. Per ciascuno sono indicati il problema affrontato, le classi che lo realizzano, i vantaggi ottenuti e l'alternativa scartata.

5.1 Dependency Injection

- **Problema.** I servizi remoti sono istanziati dal contenitore servlet, che sa invocare solo il costruttore senza argomenti. Se ciascuno costruisse al proprio interno la classe di accesso ai dati, ogni test avrebbe dovuto passare per il database su file, con esecuzioni lente, interdipendenti e capaci di sporcare i dati di lavoro.
- **Realizzazione.** Ogni servizio interessato espone due costruttori: quello senza argomenti, usato dal contenitore, che costruisce la dipendenza reale, e uno che la riceve dall'esterno, usato dai test. Vale per `RecensioneServiceImpl`, `ReputazioneServiceImpl`, `ChatServiceImpl` e `RichiestaScambioServiceImpl`. Lo stesso schema è applicato un livello più sotto da `AnnuncioDatabase(DB)` e `RichiestaScambioDatabase(DB)`, che accettano il database su cui lavorare.
- **Vantaggi.** `ChatServiceImplTest` e `RichiestaScambioServiceImplTest` costruiscono il servizio su un database in memoria e verificano le regole senza avviare né contenitore né applicazione. Le dipendenze di ogni servizio sono leggibili nella firma del costruttore invece che sparse nel corpo dei metodi. Aggiungere un caso di test non richiede di ripulire lo stato lasciato dai precedenti.
- **Alternativa scartata.** Un framework di iniezione come Guice o Spring: avrebbe introdotto una dipendenza e una configurazione sproporzionate rispetto a otto servizi, e si sarebbe dovuto integrare con l'istanziamento delle servlet, che resta in mano al contenitore. Il doppio costruttore ottiene lo stesso beneficio sui test con due righe di codice.

5.2 CONTROLLER (GRASP)

- **Problema.** Dodici schermate devono poter eseguire operazioni di sistema senza conoscere come i dati sono memorizzati, e senza che ciascuna reimplementi le regole a modo proprio.
- **Realizzazione.** Ogni area funzionale ha il proprio controller lato server: `AccessoServiceImpl`, `RegistrazioneServiceImpl`, `ProfileServiceImpl`, `AnnuncioServiceImpl`, `MarketplaceServiceImpl`, `RichiestaScambioServiceImpl`, `ChatServiceImpl`, `RecensioneServiceImpl`, `ReputazioneServiceImpl`. Ciascuno riceve le operazioni di un insieme coeso di casi d'uso e le coordina.
- **Vantaggi.** Le *Gui dipendono dall'interfaccia del servizio e non dalle classi di persistenza: cambiare il modo in cui gli annunci sono salvati non tocca `MarketplaceGui`. I controlli che non possono essere delegati al client stanno in un punto solo e sono verificabili da un test: `RichiestaScambioServiceImpl` ricava il proprietario dell'annuncio lato server e rifiuta le richieste su un proprio annuncio, quindi un client manomesso non può falsificare l'identità del creatore. La suddivisione per area rende raro il conflitto quando più persone lavorano in parallelo.
- **Alternativa scartata.** Un unico servizio remoto per l'intera applicazione: sarebbe stato più semplice da configurare, ma con una coesione bassissima, con un solo file toccato da tutti i membri del gruppo e con una superficie di test difficile da isolare.

5.3 INFORMATION EXPERT (GRASP)

- **Problema.** Stabilire dove collocare validazioni, filtri e valori calcolati: nella schermata che li mostra, nel servizio che li trasporta o nella classe che possiede i dati.
- **Realizzazione.** Le responsabilità stanno nelle classi che detengono le informazioni necessarie. UtenteDatabase conosce gli utenti e quindi verifica formato dell'email, robustezza della password, unicità e credenziali; AnnuncioDatabase conosce gli annunci e quindi verifica campi obbligatori, proprietà e disponibilità; RecensioneDatabase possiede le recensioni e quindi calcola la valutazione media; MessaggioDatabase sa a quale richiesta appartiene ogni messaggio e quindi decide chi può leggerlo; RichiestaScambioDatabase governa le transizioni di stato.
- **Vantaggi.** I servizi restano sottili e leggibili: AnnuncioServiceImpl è poco più di un inoltro, e questo rende evidente dove cercare una regola. La stessa regola non è duplicata tra servizi diversi che toccano gli stessi dati. I test più numerosi del progetto colpiscono direttamente queste classi, senza attraversare il livello remoto.
- **Alternativa scartata.** Validare nelle *Gui, come nel primo abbozzo della registrazione: i controlli lato client sono aggirabili e non valgono come garanzia. La regola sulla password resta comunque duplicata in RegistrazioneGui, ma con uno scopo diverso, cioè dare un messaggio immediato senza attendere il giro sul server; la verifica che conta è quella in UtenteDatabase.

5.4 PURE FABRICATION (GRASP)

- **Problema.** Alcune logiche non appartengono a nessuna entità del dominio — non sono l'utente, non sono l'annuncio, non sono la recensione — ma servono in più punti e, lasciate dove capita, diventano non riusabili e non testabili.
- **Realizzazione.** Sono state introdotte classi artificiali dedicate: ValidatoreFotoProfilo per le regole di accettazione delle immagini, ArchivioFotoProfilo per la scrittura e la lettura dei file, Stelle per la rappresentazione dei voti, ProfiloVisibilita per la decisione sulla sola lettura, RecensioneItem per il riquadro di una recensione. Anche le stesse classi *Database sono di questa natura.
- **Vantaggi.** ValidatoreFotoProfilo non dipende né da servlet né da filesystem, quindi ValidatoreFotoProfiloTest verifica tutte le combinazioni di nome, tipo e dimensione senza avviare nulla; lo stesso vale per StelleTest e ProfiloVisibilitaTest. RecensioneItem ha eliminato la duplicazione tra la schermata delle recensioni di un annuncio e il profilo pubblico, che ora restano identiche per costruzione: un ritocco grafico si fa una volta sola.
- **Alternativa scartata.** Tenere le regole di validazione dentro FotoProfiloUploadServlet e il codice di disegno dentro ciascuna schermata. Sarebbe stato più immediato, ma le regole sarebbero state verificabili solo inviando richieste HTTP con file veri, e le due schermate delle recensioni sarebbero divergute alla prima modifica.

5.5 COMMAND (GRASP/GOF) — LA BARRA DI NAVIGAZIONE

- **Problema.** NavBar deve costruire le voci di menu e gestire l'evidenziazione della sezione corrente senza conoscere le schermate che apre, e deve permettere di aggiungerne di nuove senza essere riscritta.
- **Realizzazione.** L'interfaccia NavBar.Comando dichiara il solo metodo esegui(). Ogni voce è registrata con aggiungiSezione(nome, comando), dove il comando è l'azione di apertura della schermata; la barra si limita a invocarlo al click, senza sapere cosa faccia.
- **Vantaggi.** Aggiungere una sezione è una riga sola nel costruttore. NavBar non contiene alcuna condizione sul nome della voce per decidere cosa aprire, quindi non va modificata quando

l'applicazione cresce. La logica della voce attiva, che non è cliccabile perché si è già su quella schermata, è scritta in un punto unico e vale per tutte le voci presenti e future.

- **Alternativa scartata.** Uno switch interno alla barra sul nome della voce selezionata: ogni nuova schermata avrebbe richiesto di modificare NavBar, con il rischio di dimenticare un ramo, e avrebbe legato la barra a tutte le classi di interfaccia dell'applicazione.

5.6 PROTECTED VARIATIONS (GRASP) — I CONTRATTI CONDIVISI

- **Problema.** Client e server devono accordarsi su alcune convenzioni che il compilatore non può verificare: i nomi dei campi del form di caricamento, i prefissi delle risposte, il limite di dimensione, l'insieme dei campi su cui si può cercare. Scritte due volte, queste convenzioni divergono in silenzio e l'errore emerge solo a esecuzione.
- **Realizzazione.** ProtocolloUploadFoto, collocata tra le classi condivise, è l'unico posto in cui il contratto di caricamento è scritto: percorso della servlet, nome del campo file, nome del campo email, prefissi OK| ed ERRORE|, limite di 2 MB. Sia il form lato client sia FotoProfiloUploadServlet leggono da lì, e ValidatoreFotoProfilo riusa la stessa costante di dimensione. Allo stesso modo l'enum CampoRicerca è l'unica definizione dei campi ricercabili, usata dal client per comporre la selezione e dal server per applicarla.
- **Vantaggi.** Portare il limite delle immagini da 2 a 5 MB significa cambiare una costante. Aggiungere un campo di ricerca significa aggiungere un valore all'enum, e il compilatore segnala il punto del server rimasto da aggiornare. Con l'enum è impossibile chiedere una ricerca su un campo inesistente: l'errore diventa impossibile invece che raro.
- **Alternativa scartata.** Ripetere le stringhe letterali nelle due parti. È la soluzione che non richiede alcuna classe in più, ma un refuso in una delle due copie produce un caricamento che fallisce senza spiegazione, e nessun test lo intercetta.

6. CREDENZIALI DEGLI UTENTI DEMO

6.1 STATO ATTUALE

- Il database viene popolato con un insieme completo di dati dimostrativi dalla classe DatabaseSeeder, così che la piattaforma sia già navigabile al primo avvio.
- Il popolamento non si limita agli utenti: crea anche annunci, richieste di scambio in tutti gli stati previsti, messaggi di chat e recensioni, in modo che ogni funzionalità sia dimostrabile senza doverla prima costruire a mano.
- Tutti gli account condividono la stessa password, Demo1234!, per comodità in sede di prova. Come ogni altro account, le password sono salvate come impronta SHA-256: il seeder passa dalla procedura di registrazione ordinaria, non scrive direttamente nel database.
- Il popolamento è **ripetibile senza rischi**: se i dati demo sono già presenti, un secondo lancio non fa nulla e non li duplica (§6.4).

6.2 ACCOUNT DEMO DISPONIBILI

Password valida per tutti: **Demo1234!**

Email (username)	Nome	Competenze	Annuncio pubblicato
demo1@skillshare.it	Giulia Ferrari	Programmazione Java, Spring Boot, Git	Ripetizioni di Java e Spring Boot

Email (username)	Nome	Competenze	Annuncio pubblicato
demo2@skillshare.it	Marco Bianchi	Lingua inglese, Traduzione, Public speaking	Conversazione in inglese B2/C1
demo3@skillshare.it	Sofia Greco	Fotografia, Adobe Lightroom, Video editing	Ritratti fotografici e post-produzione
demo4@skillshare.it	Luca Moretti	Chitarra acustica, Teoria musicale, Produzione audio	Lezioni di chitarra acustica per principianti

- Ogni account ha biografia, foto segnaposto e tre competenze già compilate; ogni annuncio è coerente con le competenze del suo autore.
- Il progetto non prevede ruoli differenziati: non esiste un campo che distingua un amministratore da un utente ordinario. Tutti gli account hanno le stesse capacità e le differenze dipendono solo da quali annunci e richieste appartengono a ciascuno.

6.3 COSA SI PUÒ PROVARE CON CIASCUN ACCOUNT

I quattro account sono collegati tra loro in modo da coprire l'intero ciclo di vita di uno scambio.

- **demo1@skillshare.it (Giulia Ferrari)** — l'account più completo, coinvolto in tre scambi.
 - Riceve una richiesta *in attesa* da demo2: si possono provare **Accetta** e **Rifiuta**.
 - Ha uno scambio *accettato* con demo3, con chat attiva.
 - Ha ricevuto una recensione da demo3 (**4 stelle**), visibile sul profilo pubblico.
 - Il suo annuncio ha uno scambio accettato in corso, quindi **Modifica**, **Elimina** e **Sospendi** risultano disabilitati: è il comportamento previsto per gli annunci impegnati.
- **demo2@skillshare.it (Marco Bianchi)** — utile per vedere un profilo **senza valutazione**.
 - Ha inviato una richiesta *in attesa* a demo1.
 - Ha rifiutato una richiesta ricevuta da demo4.
- **demo3@skillshare.it (Sofia Greco)** — il profilo migliore per verificare le recensioni.
 - Ha uno scambio *accettato* con demo1, con chat attiva.
 - Ha uno scambio *completato* con demo1 e ha ricevuto una recensione (**5 stelle**).
- **demo4@skillshare.it (Luca Moretti)** — il suo annuncio è **libero da scambi**, quindi è quello giusto per provare **Modifica**, **Elimina** e **Sospendi** su un annuncio disponibile.
 - Ha inviato una richiesta a demo2, che l'ha rifiutata.

Dati complessivi creati dal seeder: 4 utenti, 4 annunci, 4 richieste (una per stato: *in attesa*, *accettata*, *rifiutata*, *completata*), 5 messaggi e 2 recensioni reciproche sullo scambio completato.

6.4 NOTA SULLE FOTO DEMO

Le foto dei profili demo puntano a un servizio esterno di immagini segnaposto e richiedono connessione a Internet. In sua assenza compare l'avatar con le iniziali: è il comportamento di ripiego previsto, non un errore.

6.5 CREARE ACCOUNT AGGIUNTIVI PER IL COLLAUDO

Oltre agli account demo, se ne possono creare altri dalla schermata di registrazione. Vincoli applicati:

- email in formato valido e non già registrata;

- password di almeno 8 caratteri, con almeno una maiuscola, una minuscola, una cifra e un simbolo tra @\$%^&+=!.